```
      method: "POST",
      body: formData,
      headers: {
        "X-Requested-With": "XMLHttpRequest",
        "X-CSRF-Token": csrfToken,
        credentials: "same-origin",
      },
    })
    dispatch({ type: "addFavorite", concert })
  }
}

export const removeFavorite = (concert: Concert): ScheduleThunk => {
  return async (dispatch) => {
    const formData = new FormData()
    await fetch(`/favorites/${concert.id}.js`, {
      method: "DELETE",
      body: formData,
      headers: {
        "X-Requested-With": "XMLHttpRequest",
        "X-CSRF-Token": csrfToken,
        credentials: "same-origin",
      },
    })
    dispatch({ type: "removeFavorite", concert })
  }
}
```

The pattern here is to simulate a DOM FormData object with the concert ID of the new favorite and trigger an API call to Rails. We don't need the user ID because Rails is still managing a session, but we do need the Rails csrf-token, which is in the header and used by Rails to prevent cross-site scripting attacks.

Once the form is submitted, both thunks dispatch to their respective actions:

appendix_react/01/app/javascript/contexts/schedule_context.ts

```
case "addFavorite": {
  return {
    ...state,
    mostRecentFavoriteId: action.concert.id,
    favorites: [...state.favorites, action.concert].sort(
      (a, b) => a.sortDate - b.sortDate
```